Comparison of algorithms used in single-cell transcriptomic data analysis

Jafar Isbarov[1], Elmir Mahammadov[2,3,4]

[1] Azerbaijan State Oil and Industry University, Baku, Azerbaijan
[2] Institute of Computational Biology, Helmholtz Zentrum München, Neuherberg, Germany
[3] Institute of Epigenetics and Stem Cells, Helmholtz Zentrum München, Munich, Germany
[4] Institute of Functional Epigenetics, Helmholtz Zentrum München, Neuherberg, Germany

Abstract

Single-cell analysis is an increasingly relevant approach in "omics" studies. In the last decade, it has been applied to various fields, including cancer biology, neuroscience, and, especially, developmental biology. This rise in popularity has been accompanied with creation of modern software, development of new pipelines and design of new algorithms. Many established algorithms have also been applied with varying levels of effectiveness. Currently, there is an abundance of algorithms for all steps of the general workflow. While some scientists use ready-made pipelines (such as Seurat), manual analysis is popular, too, as it allows more flexibility. Scientists who perform their own analysis face multiple options when it comes to the choice of algorithms. We have used two different datasets to test some of the most widely-used algorithms. In this paper, we are going to report the main differences between them, suggest a minimal number of algorithms for each step, and explain our suggestions. In certain stages, it is impossible to make a clear choice without further context. In these cases, we are going to explore the major possibilities, and make suggestions for each one of them.

Introduction

Quantitative analysis of cell contents - mostly DNA, RNA and protein - has been a major part of molecular biology since the emergence of the field. Studying the protein content of the cell is helpful when trying to uncover dynamics of its metabolism. Observing changes in DNA sequence through the generations allows us to understand the evolutionary processes at a cellular level. Comparing the amount of different RNA molecules enables us to fill the gap between these two separate pieces of information by unveiling the expression patterns of the cell.

Our report concentrates on single-cell RNA sequencing analysis - a revolutionary combination of molecular and computational techniques that opens up new dimensions in our understanding of tissues and cell types via more detailed analyses of spatial and temporal variations in gene expression. Before we get started, it is important to understand both conceptual and technical differences between single-cell analysis and its traditional counterpart.

The tradition: bulk analysis

Molecular analyses rely on sample abundance. In order to sequence a single gene, we need multiple copies. The same thing applies to RNA sequences, too. Amplification techniques such as PCR allows us to sequence small DNA and RNA samples. We can reproduce and sequence particular nucleic acids, but we can not reproduce individual cells. Instead, a tissue has to be assumed to be homogeneous and analyzed in bulk. The power of this approach lies in its simplicity - so does its weakness. Bulk analysis did, and still does, provide insightful information about expression levels of genes in a tissue. But this strips us of our ability to analyze cells individually. This limitation is not merely principal. It comes with a set of problems in our understanding of cell states. Simpson's Paradox is an interesting example of this. It refers to situations in statistical analysis where a trend is observed in separate groups of data, but reverses or disappears altogether when the said groups are combined. For example, expression levels of two genes that otherwise look negatively correlated may turn out to have positive correlation after proper clustering [Trapnell, 2015].

Only in the 21st century did we begin to approach these problems. Progress in molecular techniques, coupled with advances in computer science and information technologies set the stage for a new generation of analysis tools to be developed.

The novelty: single-cell analysis

As a result of progress in amplification techniques and cell capture/isolation instruments, we are now able to perform RNA sequencing on individual cells. RNA-seq is not the only relevant method of measuring cell state; others include Hi-C (method for measuring chromatin conformation), which is useful for determining the physical interaction of genome regions [Hakim and Misteli, 2012]; bisulfite sequencing, and DNase I hypersensitivity sequencing [Trapnell, 2015]. Each one of these can now be performed on a single-cell scale. However, single-cell RNA sequencing analysis has been our sole concentration through the program, as others are more expensive and can have lower sensitivity. Before reporting our research and findings, we first provide a brief introduction to the field.

Single-cell sequencing

Transcriptome of the cell has to be sequenced before we can begin computational analysis. Single-cell sequencing is composed of the following steps: (i) cell isolation; (ii) cell lysis; (iii) reverse transcription; (iv) amplification; (v) transcript coverage; (vi) strand specificity; and (vii) Unique Molecular Identifiers (Chen et al., 2019). There are several platforms that enable RNA (and cDNA) sequencing at single-cell resolution. They can be divided into two groups: (1) molecular tag-based methods and (2) full-length methods. 10x Genomics Chromium system and MARS-seq belong to the first group. They are both cheaper, faster, and allow sequencing of more cells at the same time, but provide limited coverage of the transcriptome. The second group, which includes SMART-seq and SMART-seq2, covers the entire transcriptome, but these methods come with immense time and money costs. Additionally, they allow sequencing of a lower number of cells at the same time. Both types of platforms have their place in single-cell studies, however, due to their lower cost and higher speed, molecular tag-based methods are used more widely [See et al., 2018].

Stages of single-cell RNA sequencing (scRNA-seq) analysis

scRNA-seq analysis can be divided into two main stages: pre-processing and downstream analysis. Raw data has to pass through QC to assure no doublets and "bad" cells remain. If data comes from different experiments, batch correction has to be performed to account for the so-called batch-effect. Data (especially, count matrix) also have to be scaled to ease clustering and other related processes. Final stage of pre-processing is feature selection. Every gene represents a feature and using all of them in our analysis would slow it down considerably [Luecken and Theis, 2019].

Once the data has been pre-processed, it is ready for downstream analysis. Initial stage is clustering, marker identification, and cluster annotation. After this, data is ready for trajectory inference, differential expression and compositional analysis. Trajectory inference offers a dynamic, temporal understanding of gene expression. Compositional analysis is essentially about changes in compositions - cell-identity clusters. Differential expression analysis allows us to see the variety in gene expression across the tissue(s). Downstream analysis does not necessarily include every single one of these stages, although research papers usually include more than one [Luecken and Theis, 2019]. Figure 1 gives a detailed illustration of a typical workflow.

Analyzed Papers

Single-cell analysis has evolved into a considerably large field. As of April, 2020, there are more than 1200 articles in the Single Cell Studies Database. Due to limited time and resources at hand, we picked only two papers from this database, taking into account several factors:

- The paper has to involve single-cell RNA sequencing analysis as one of its major tools of research. That is to say, scRNA-seq analysis has to lead to major findings in the context of the paper.
- Most, if not all stages of the scRNA-seq analysis pipeline has to be present in the research. Virtually no single research utilizes every single technique of scRNA-seq analysis, so it is more reasonable to choose our two papers to maximize the coverage.
- Single cell data have to be available preferably in raw count matrix format. Count matrix, observations (cells) and variables (genes) constitute the backbone of the scRNA-seq data, no analysis can be performed if even one of them is missing. Cell and gene annotation provide more opportunities for analysis, but they are not imperative.
- The tools and platforms used in scRNA-seq analysis have to be documented clearly. We should be able to replicate some of the results presented in the paper (e.g. cell clustering). However, large-scale, especially machine learning algorithms tend to have a stochastic element to them, which means that in order to get some sort of similarity between our results, we need to use exactly the same tools. Otherwise we can end up with virtually distinct outcomes.

Vastness of the database allowed us some freedom of choice, which means our preferences also had considerable effect in the choice of papers. We ended up with these two:

1. *[Sharma et al., 2018]:* In this paper, authors use scRNA-seq to probe variants of chemo-resistance in tumor cells. Sequencing was performed using the C1 Single-Cell Auto Prep IFC (Fluidigm) system. Data has been submitted to NCBI in pre-processed and QC-ed form. Annotations are available for cells, but not for

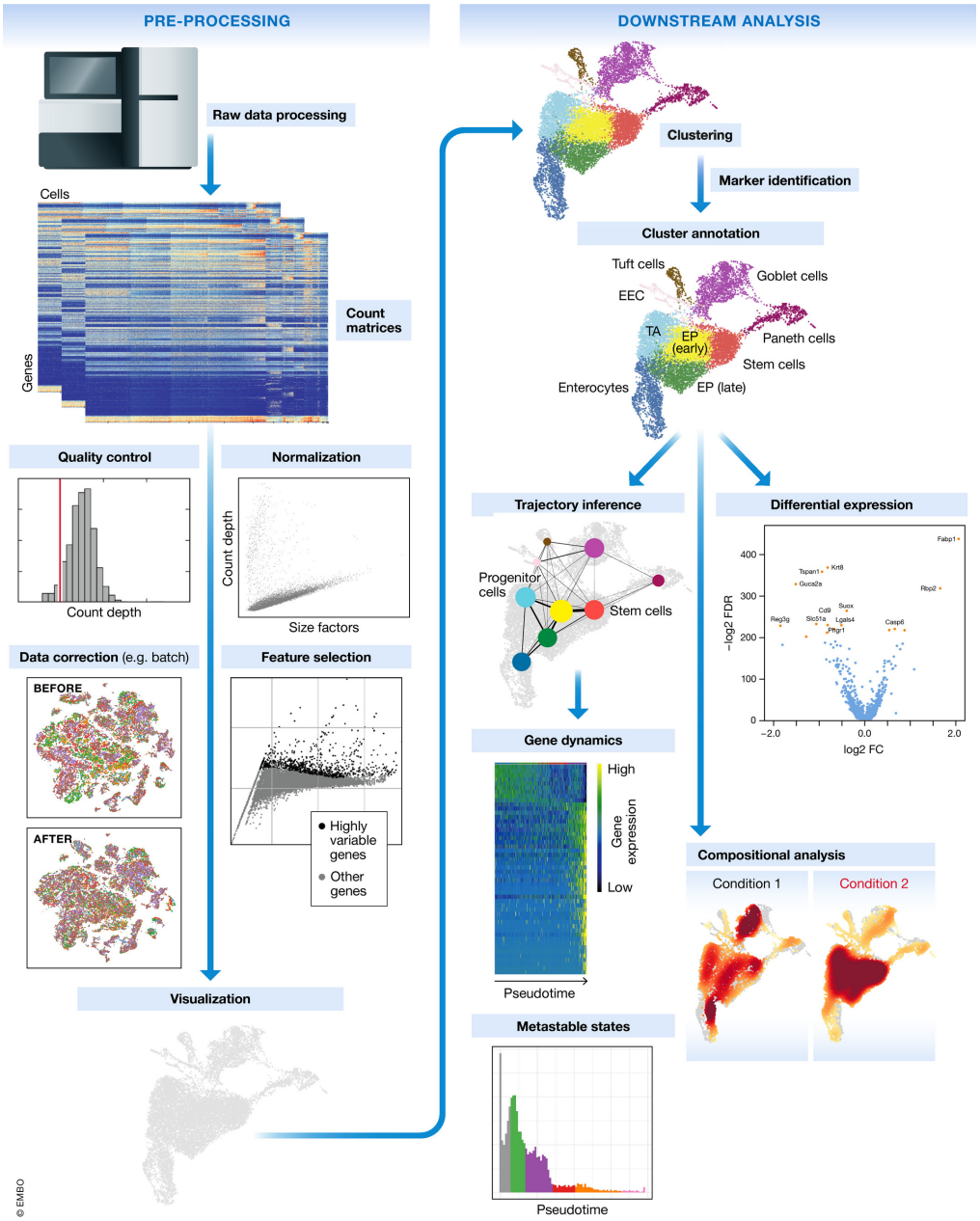


Figure 1. Diagram of a typical single-cell RNA-seq analysis pipeline [Luecken and Theis, 2019]

genes. Cell annotations include cell color, patient ID, origin, drug status, and clustering labels.

2. *[Johnson et al., 2020]:* In this paper, authors use scRNA-seq to classify cells found in homeostatic (where there is no need for regeneration) and regenerating mouse digit tips. Sequencing was performed on 10x Genomics Chromium platform. The data has been submitted to NCBI in raw form. We are given 5 different datasets for each stage (each tissue). No annotation is available for genes. However, cell types have been added as supplementary data.

Results

[Sharma et al., 2018]

In this article, we are provided with pre-processed count matrix and cell annotations. Therefore, pre-processing algorithms were not tested. Instead, we concentrated on dimensionality-reduction and clustering algorithms. Various clustering metrics were used to compare results of the clustering algorithms. Three different algorithms were probed for dimensionality-reduction:

1. Principal Component Analysis (PCA)
2. Uniform Manifold Approximation and Projection (UMAP)
3. t-distributed Stochastic Neighbor Embedding (tSNE)

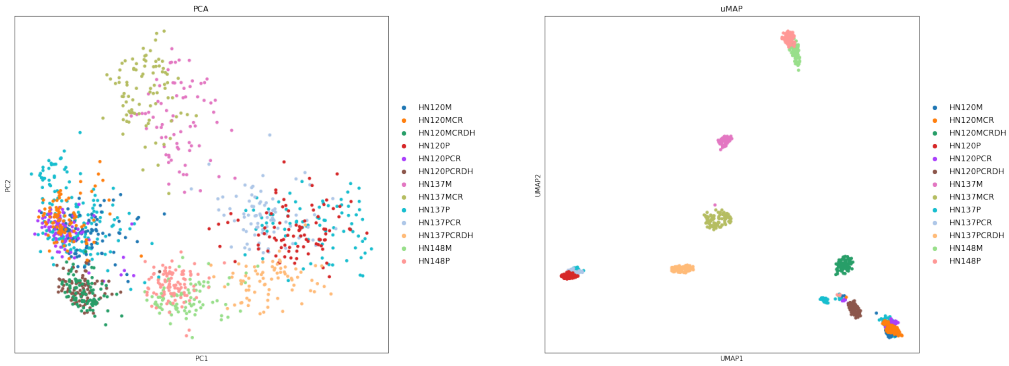
It was discovered that UMAP and tSNE create more distinct clusters, which is useful for visualization purposes, while PCA clusters were more diffuse, hence harder to visually keep track of. We were unable to test whether this apparent advantage of UMAP and tSNE comes with more information loss as there is currently no method to measure information loss for UMAP and tSNE (figure 2).

Two different clustering methods were used and their results compared. The first method was k-means (centroid-based) clustering. Principal component analysis was performed on the data, after which scikit-learn implementation of k-means algorithm [Pedregosa et al., 2018] was applied to it. The second method was graph clustering. We used the Leiden and Louvain algorithms to perform the clustering [Traag et al., 2019]. We performed comparisons between these two methods (centroid-based vs. graph clustering), between different algorithms of the same method (Leiden vs. Louvain), as well as between different setups of the same algorithm (k-means clusterings with vs. without PCA).

We initially performed two clusterings using the scikit-learn implementation of k-means algorithm ($K = 5$). One of the clusterings was performed on principal components, while the second clustering was performed on the original data. The results did not differ significantly (See tables 1 and 2). We then performed the same comparison among the next two clustering, where we assigned $K = 25$. Increasing the number of clusters widened the gap between PC and non-PC clusterings.

After this, we compared results of Leiden and Louvain algorithms. The Leiden algorithm has been designed based on the Louvain algorithm. It offers faster and more accurate performance [Traag et al., 2019]. The difference between the obtained results was not significant enough to reject the null hypothesis (See table 3). (This single case cannot be interpreted as an exhaustive evaluation of Leiden and Louvain algorithms.)

We had to perform preliminary analysis to choose the resolution value for Leiden and Louvain algorithms. Comparing Silhouette score [Rousseeuw, 1987] with resolution was not useful, as plotting the Silhouette score does not result in an elbow point.



(a) PCA

(b) UMAP

(c) tSNE

Figure 2. Comparison of dimensionality-reduction algorithms. (Coloring represents cell groups.)

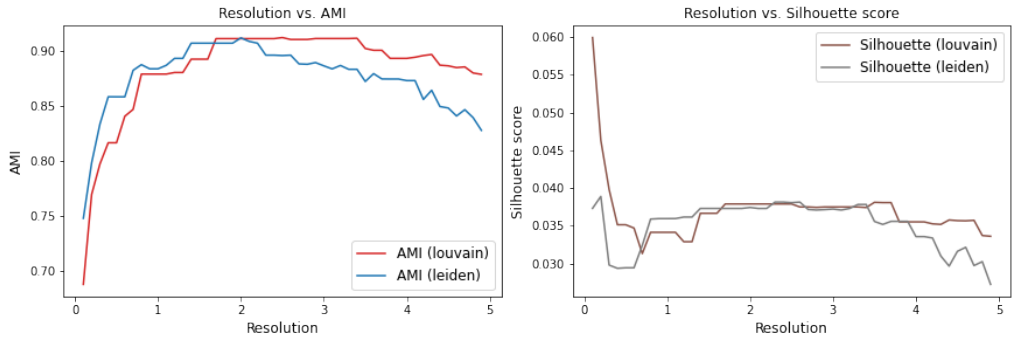


Figure 3. Finding the appropriate resolution for graph-clustering algorithms.

Metric	**Range**	**Result**
Adjusted Rand Index	[0,1]	0.9959
Jaccard Index	[0,1]	0.6237
Adjusted Mutual Information	[0,1]	0.9933
Silhouette Coefficient (PCA)	[-1,1]	0.0588
Silhouette Coefficient	[-1,1]	0.0590
Calinski-Harabasz Index (PCA)	[-,-]	63.4060
Calinski-Harabasz Index	[-,-]	63.4053

Table 1. Comparison of two different k-means clustering results ($K = 5$). The first clustering has been performed on principal components, while the second clustering has been performed on the original data.

Instead, we choose Adjusted Mutual Information score [Vinh et al., 2009], which proved more useful in determining necessary resolution (figure 3).

Lastly, we compared results of centroid-based and graph clustering methods. We selected K to be 25 for the k-means algorithm. We preferred the Leiden algorithm over Louvain, as the former is known to perform better [Traag et al., 2019]. Both clusterings were performed on the principal components.

[Johnson et al., 2020]

In this paper, we are provided with raw count matrix. Single-cell sequencing has been performed on the 10x Genomics Chromium system (Single Cell 3' v2 and v3 chemistry), which means its coverage is limited. We compared three data visualization methods for marker gene and cell type identification:

1. Dimensionality-reduction and scatter plot

Metric	**Range**	**Result**
Adjusted Rand Index	[0,1]	0.6844
Jaccard Index	[0,1]	0.0891
Adjusted Mutual Information	[0,1]	0.8281
Silhouette Coefficient (PCA)	[-1,1]	0.0273
Silhouette Coefficient	[-1,1]	0.0273
Calinski-Harabasz Index (PCA)	[-,-]	18.4607
Calinski-Harabasz Index	[-,-]	18.3921

Table 2. Comparison of two different k-means clustering results ($K = 25$). The first clustering has been performed on principal components, while the second clustering has been performed on the original data.

Metric	Range	Result
Adjusted Rand Index	[0,1]	0.9652
Jaccard Index	[0,1]	0.6171
Adjusted Mutual Information	[0,1]	0.9688
Silhouette Coefficient (PCA)	[-1,1]	0.0359
Silhouette Coefficient	[-1,1]	0.0341
Calinski-Harabasz Index (PCA)	[-,-]	30.0971
Calinski-Harabasz Index	[-,-]	31.2248

Table 3. Comparison of Leiden and Louvain clustering results.

Metric	Range	Result
Adjusted Rand Index	[0,1]	0.6904
Jaccard Index	[0,1]	0.0381
Adjusted Mutual Information	[0,1]	0.8617
Silhouette Coefficient (PCA)	[-1,1]	0.0359
Silhouette Coefficient	[-1,1]	0.0260
Calinski-Harabasz Index (PCA)	[-,-]	30.0971
Calinski-Harabasz Index	[-,-]	18.4607

Table 4. Comparison of k-means and leiden clustering results. ($K = 15$; both clusterings are performed on principal components.)

2. Violin plot
3. Heatmap

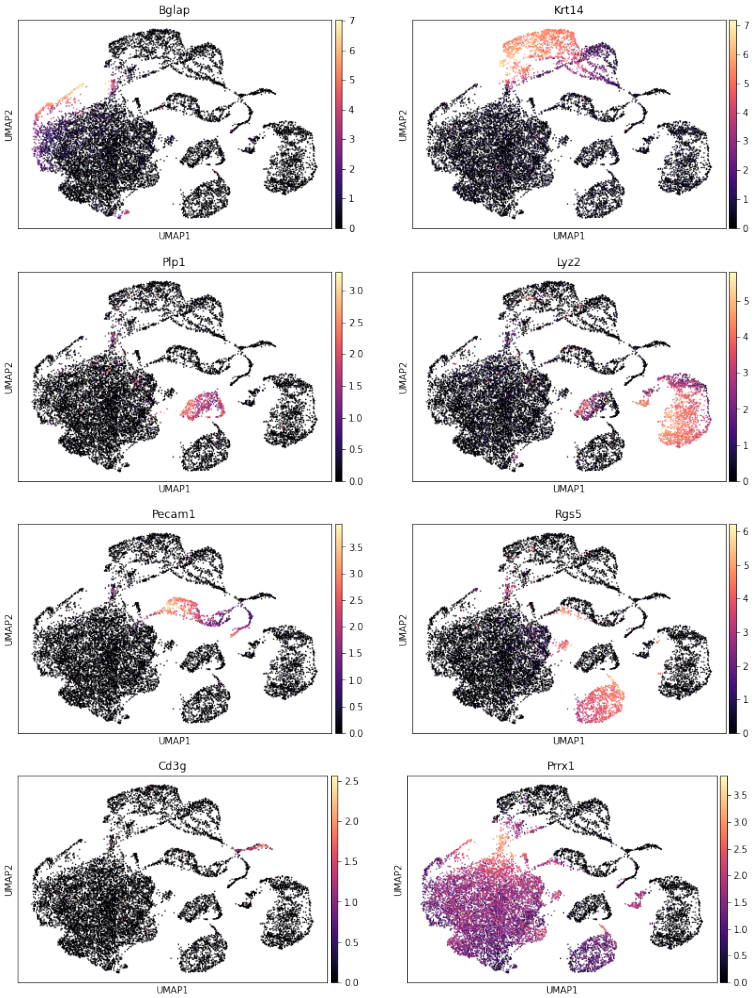
After performing Leiden and k-means clustering, the resulting clusters were displayed against expression levels of marker genes. Among the three visualization methods, dimensionality-reduction and scatter plot proved the least effective. Clusters and expression levels of marker genes had to be plotted separately, and then compared. This required too much visual examination on the part of the researcher, and relied on sharp sensory memory, therefore we consider it more prone to human error (figure 4).

Heatmap improves the situation, as it lines up each cluster against expression levels of a marker gene. But expression levels are indicated by a color scale, which makes comparison harder than it should be (figure 5). We made several mistakes with this method, which we later discovered with help of the violin plot. Violin plot also helped us to make decisions in otherwise undecidable cases.

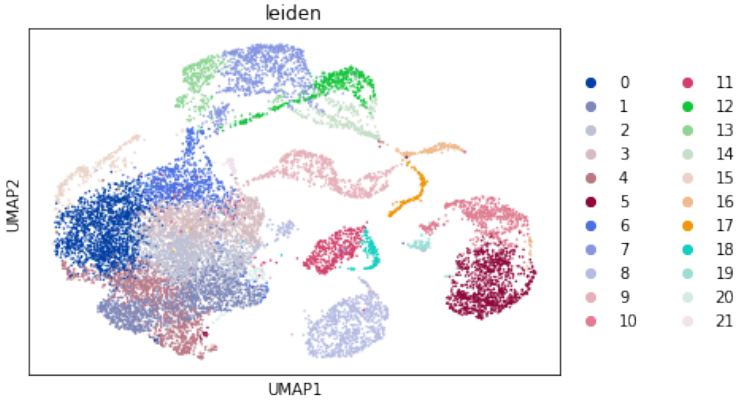
Each violin plot displays clusters against expression levels of a single marker gene. Expression level is indicated by the y axis rather than a color scale. This makes it easier to compare expression levels of a marker gene among different clusters (figure 6). Therefore, we consider violin plot the best option to identify cell types with help of marker genes. Final results of cell identification can be found in Supplementary Figures S1-S3.

Three algorithms for ranking genes for characterizing groups were also compared. These algorithms are based on three different null-hypothesis tests:

1. *Logistic regression* measures potential influence of a factor on the outcome. It allows us to find factors that have the strongest association with the outcome [Tolles and Meurer, 2016].
2. *Paired two-sample t-test* is only one of multiple t-tests. It is useful for comparison of paired samples, as it determines whether the mean difference between two sets of observations is significantly different from zero.



(a) Leiden clusters visualized with help of the UMAP algorithm.



(b) Marker genes, each displayed in individual scatterplot. Scale: expression level (log-normalized UMI).

Figure 4. Cell types are identified by matching clusters **(b)** with marker genes **(a)**. In the case of UMAP and scatter plots, it is exceptionally hard.

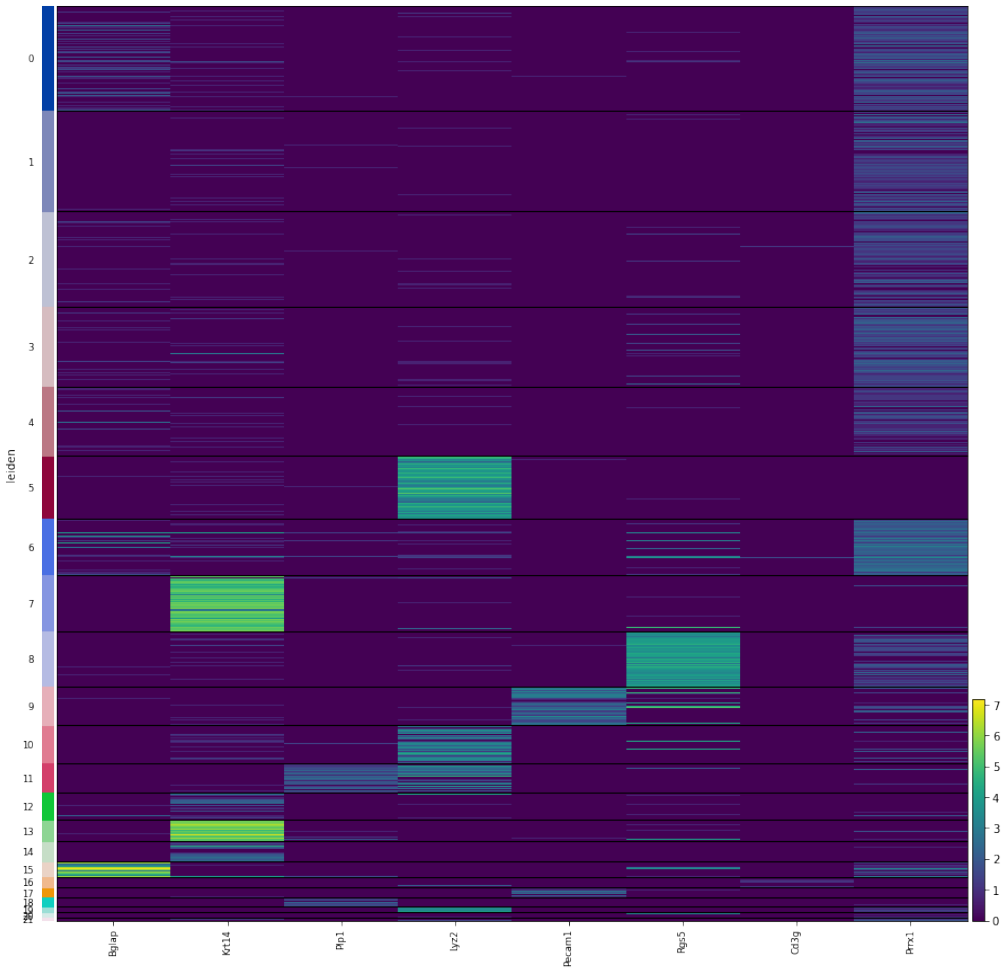


Figure 5. In the case of heatmap, matching clusters and marker genes is relatively easier. However, assigning each cluster to a single marker gene based on the expression levels still causes problems.

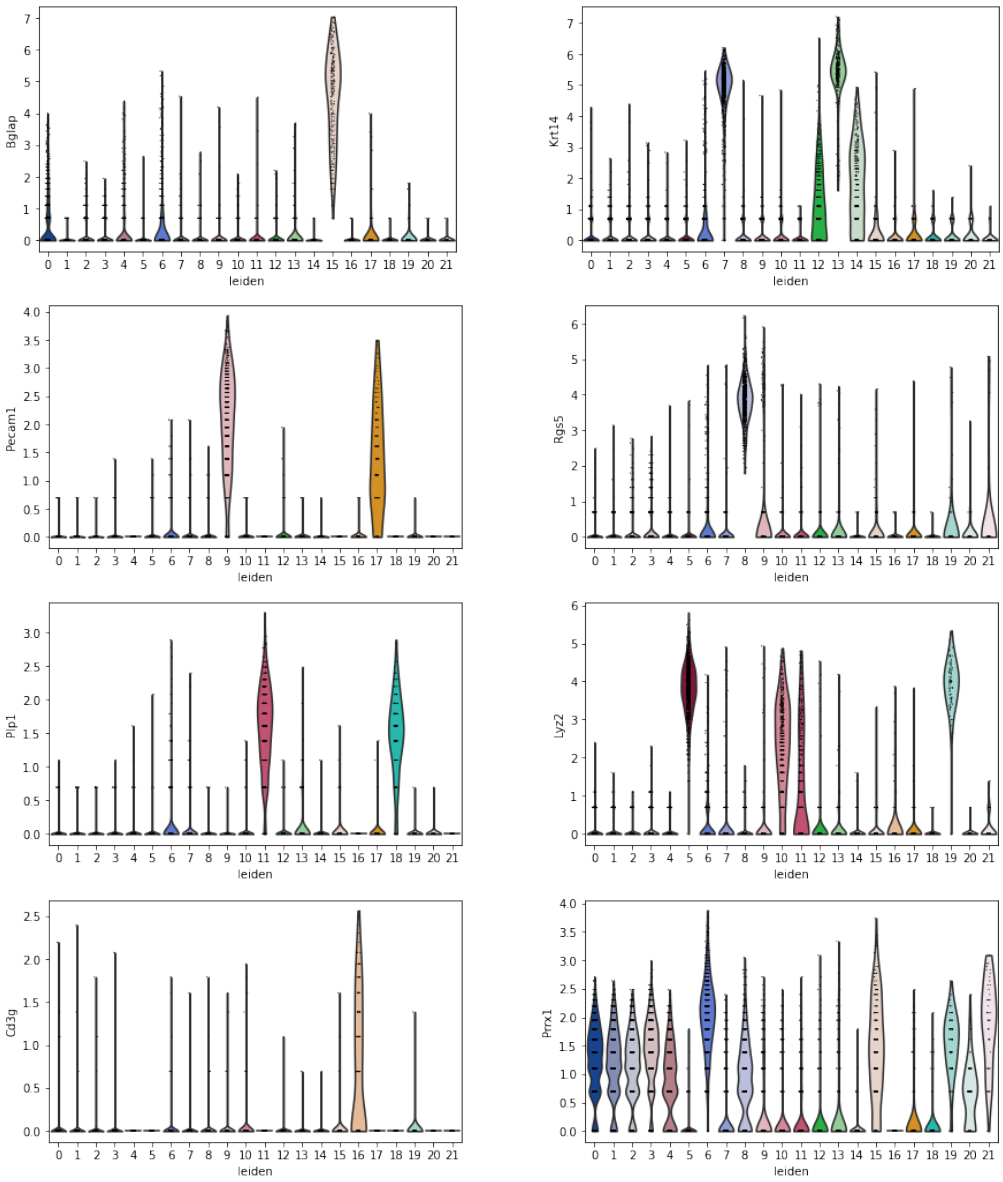


Figure 6. Violin plot decreases the workload significantly, and causes less errors.

3. *Wilcoxon rank-sum (Mann Whitney U) test* is another method used to compare two samples. It determines whether the probability of X being greater than Y is equal to the probability of Y being greater than X (X and Y being randomly selected values from two different populations).

Initially, we ranked genes for characterizing groups (here, cell types) with help of these three algorithms, and recorded the results separately. Then, we calculated the intersection of the top 100 genes for each group (cell type) computed by:

1. Wilcoxon rank-sum test vs. paired two-sample t-test
2. Wilcoxon rank-sum test vs. logistic regression
3. Paired two-sample t-test vs. logistic regression

We found out that Wilcoxon signed-rank test and paired two-sample t-test were closer to each other than to logistic regression.

Conclusion and outlook

We compared different algorithms used in several stages of single-cell RNA sequencing analysis. Main concentration was on dimensionality-reduction, clustering, visualization, and ranking genes for characterizing groups. In some cases, we were able to make clear recommendations regarding these algorithms. However, in most cases, we managed to perform only surface-level analysis, with no basis for a recommendation.

When performing dimensionality reduction for visualization purposes, UMAP or tSNE are suggested over PCA, as they result in more distinct clusters. The higher cluster count is selected in k-means clustering, the more significant effect PCA has on the result. No significant difference was found in performance of Leiden and Louvain algorithms. When comparing expression levels of marker genes and results of clustering, violin plot is suggest over both dimensionality-reduction/scatter plot and heatmap.

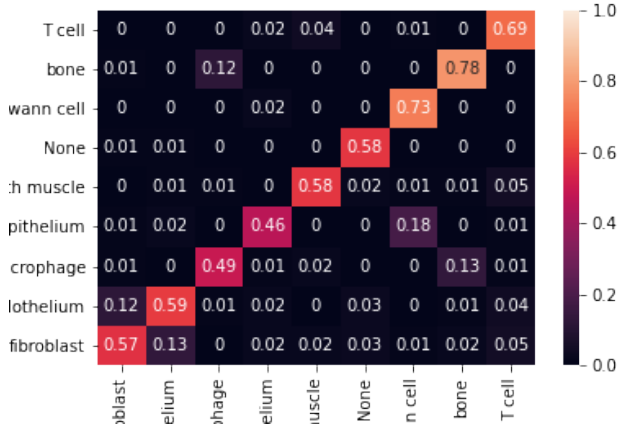
Lack of a method to measure information loss for UMAP and tSNE is serious challenge that hindered our research process. Solution of this problem would help us decide between PCA, UMAP, and tSNE algorithms for dimensionality-reduction.

Methods and tools

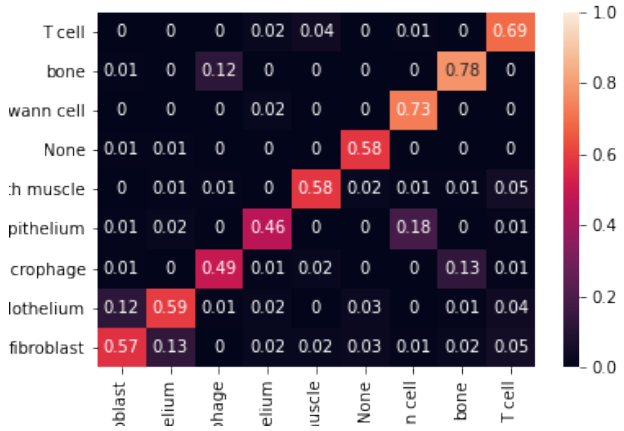
Our entire code was written in Python (v3..8.5). Scanpy (v1.7.1), scikit-learn (v0.24.1), leidenalg (v0.8.3), numpy (v1.20.2), and pandas (v1.2.3) packages have been used extensively. Biological data has been converted into an AnnData (v0.7.5) object before being processed. Other packages involved in the project are: seaborn (v0.11.1), matplotlib (v.3.4.1), and scipy (v1.6.3).

The following functions have been used in our analysis:

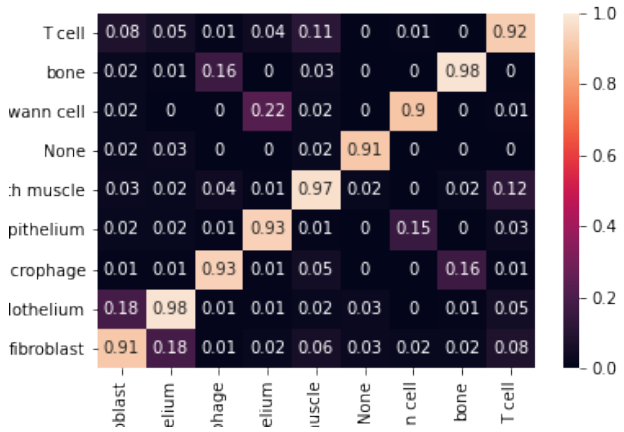
1. scanpy.pp.highly_variable_genes function has been used to find highly variable genes (n_top_genes = 3000).
2. scanpy.pp.pca (n_comps = 75 in the first paper, n_comps=16 in the second paper) and sklearn.decomposition.PCA have been used to perform PCA.
3. scanpy.pp.neighbors (n_pcs = 75 in the first paper, default in the second paper) has been used to compute neighborhood graph of observations.
4. scanpy.pl.umap, and scanpy.pl.tsne have been used for dimensionality reduction.



(a) Paired two-sample t-test vs. logistic regression.



(b) Wilcoxon rank-sum test vs. logistic regression.



(c) Wilcoxon signed-rank test vs. paired two-sample t-test.

Figure 7. Intersection of top 100 genes (ranked for characterizing cell types), computed by (a) Wilcoxon rank-sum test, (b) paired two-sample t-test, and (c) logistic regression. (Scale: intersection size)

5. sklearn.metrics.adjusted_rand_score has been used to compute adjusted rand index.
6. sklearn.metrics.jaccard_score has been used to calculate Jaccard index.
7. sklearn.metrics.silhouette_score has been used to calculate silhouette score.
8. sklearn.metrics.adjusted_mutual_info_score has been used to calculate adjusted mutual information.
9. sklearn.metrics.calinski_harabasz_score has been used to calculate Calinski-Harabasz index.
10. sklearn.cluster.KMeans (n_clusters = 5 and 25 in the first paper, n_clusters = 22, 17, 15, 19) has been used for k-means clustering.
11. sc.tl.louvain has been used to perform Louvain clustering. sc.tl.leiden (resolution = 1 in the first paper, resolution = 0.45 and 0.6 in the second paper) has been used to perform Leiden clustering.

Code and accompanying notes have been written in Jupyter Notebooks. They are available on GitHub: https://github.com/ceferisbarov/scRNA-seq

RESOURCE	SOURCE	DOCUMENTATION
Python (Version 3.8.5)	Python Software Foundation	docs.python.org
Scanpy (Version 1.7.1)	[Wolf et al., 2018]	scanpy.readthedocs .io/en/stable/api.html
Anndata (Version 0.7.5)	[Wolf et al., 2018]	anndata.readthedocs .io/en/latest/api.html
leidenalg (Version 0.8.3)	[Traag et al., 2019]	leidenalg.readthedocs .io/en/stable/intro.html
scikit-learn (Version 0.24.1)	[Pedregosa et al., 2018]	scikit-learn.org/stable/ modules /classes.html
numpy (Version 1.20.2)	[Harris et al., 2020]	numpy.org/doc/stable /reference

References

Hakim and Misteli, 2012. Hakim, O. and Misteli, T. (2012). SnapShot: Chromosome conformation capture. *Cell*, 148(5):1068–1068.e2.

Harris et al., 2020. Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., and Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825):357–362.

Johnson et al., 2020. Johnson, G. L., Masias, E. J., and Lehoczky, J. A. (2020). Cellular heterogeneity and lineage restriction during mouse digit tip regeneration at single-cell resolution. *Developmental Cell*, 52(4):525–540.e5.

Luecken and Theis, 2019. Luecken, M. D. and Theis, F. J. (2019). Current best practices in single-cell RNA-seq analysis: a tutorial. *Molecular Systems Biology*, 15(6).

Pedregosa et al., 2018. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Müller, A., Nothman, J., Louppe, G., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Édouard Duchesnay (2018). Scikit-learn: Machine learning in python.

Rousseeuw, 1987. Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65.

See et al., 2018. See, P., Lum, J., Chen, J., and Ginhoux, F. (2018). A single-cell sequencing guide for immunologists. *Frontiers in Immunology*, 9.

Sharma et al., 2018. Sharma, A., Cao, E. Y., Kumar, V., Zhang, X., Leong, H. S., Wong, A. M. L., Ramakrishnan, N., Hakimullah, M., Teo, H. M. V., Chong, F. T., Chia, S., Thangavelu, M. T., Kwang, X. L., Gupta, R., Clark, J. R., Periyasamy, G., Iyer, N. G., and DasGupta, R. (2018). Longitudinal single-cell RNA sequencing of patient-derived primary cells reveals drug-induced infidelity in stem cell hierarchy. *Nature Communications*, 9(1).

Tolles and Meurer, 2016. Tolles, J. and Meurer, W. J. (2016). Logistic regression. *JAMA*, 316(5):533.

Traag et al., 2019. Traag, V. A., Waltman, L., and van Eck, N. J. (2019). From louvain to leiden: guaranteeing well-connected communities. *Scientific Reports*, 9(1).

Trapnell, 2015. Trapnell, C. (2015). Defining cell types and states with single-cell genomics. *Genome Research*, 25(10):1491–1498.

Vinh et al., 2009. Vinh, N. X., Epps, J., and Bailey, J. (2009). Information theoretic measures for clusterings comparison. In *Proceedings of the 26th Annual International Conference on Machine Learning - ICML '09*. ACM Press.

Wolf et al., 2018. Wolf, F. A., Angerer, P., and Theis, F. J. (2018). SCANPY: large-scale single-cell gene expression data analysis. *Genome Biology*, 19(1).

Elmi Spektr Tutoring Programme is an academic programme that gives Azerbaijani students an opportunity to carry out research internships in different areas of science. For more details, visit https://elmi-spektr.com/estp/

Supplementary information

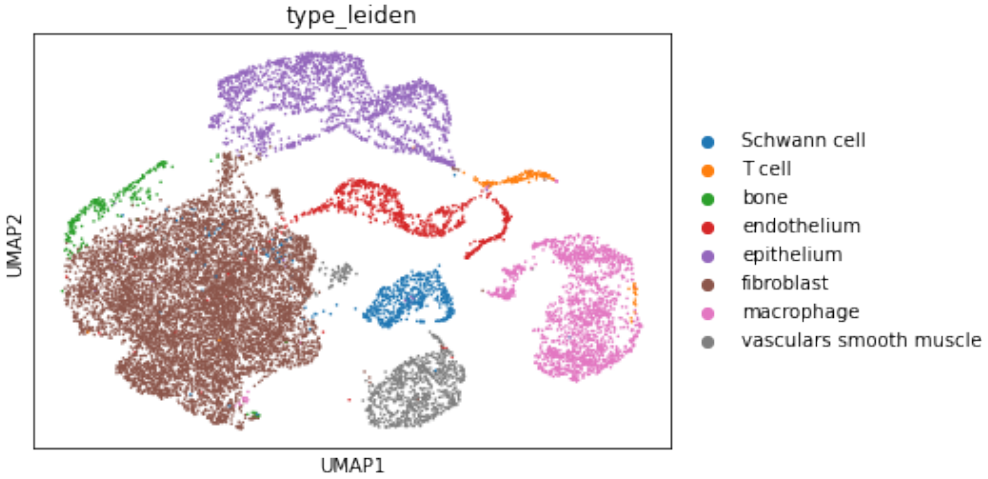


Figure S1. Visualization of predicted cell types with help of a scatter plot (UA). Cells were clustered with help of the Leiden algorithm and each cluster was assigned a cell type based on expression levels of marker genes.

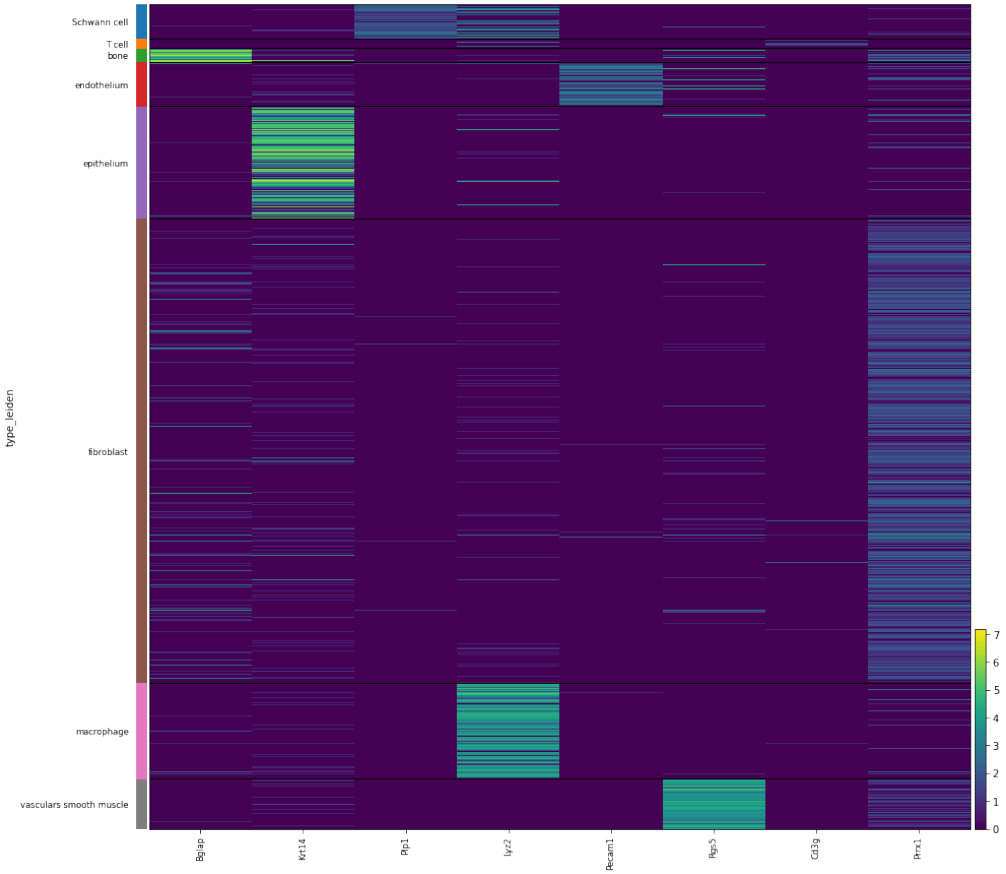


Figure S2. Visualization of predicted cell types with help of a heatmap (UA).

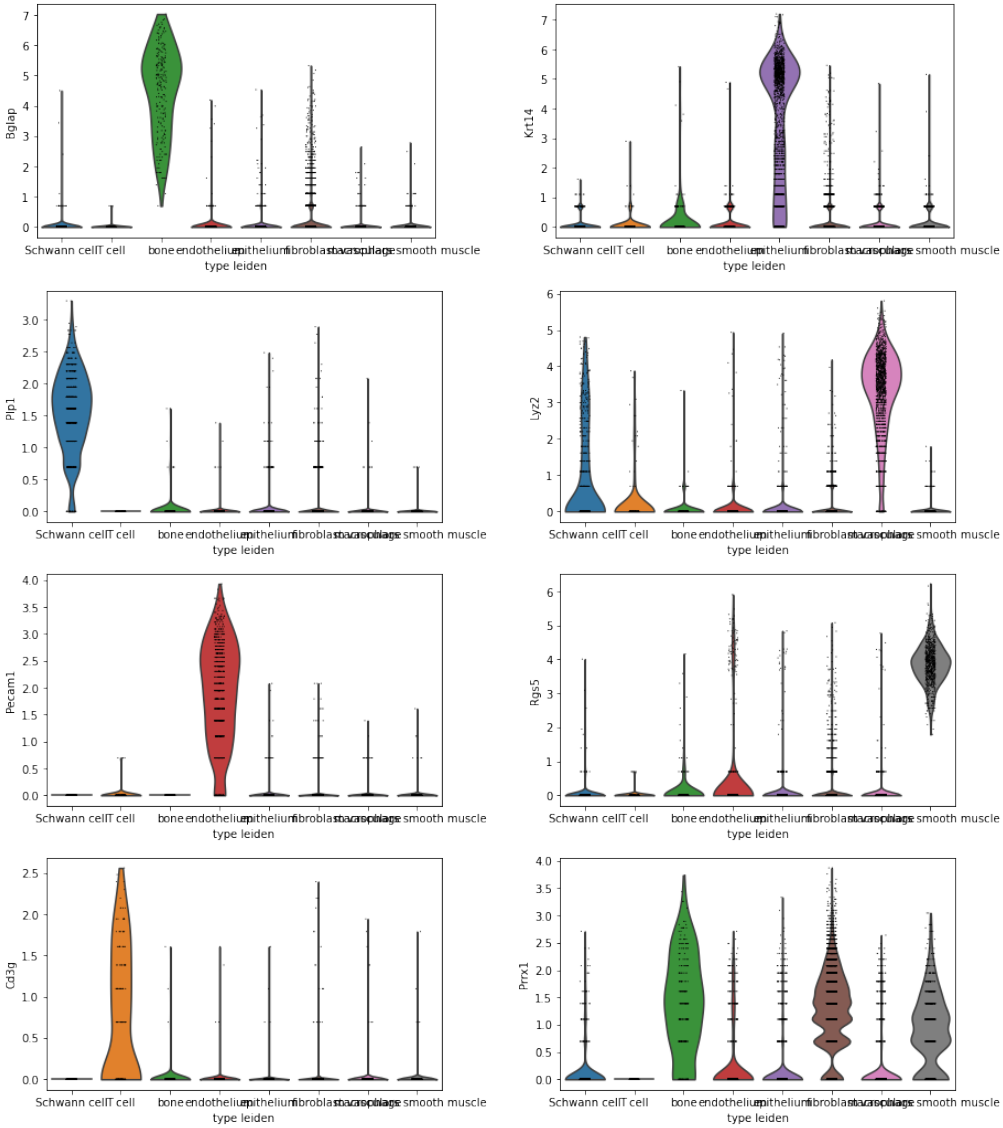


Figure S3. Visualization of predicted cell types with help of a violin plot (UA).